

Assignment 2: Number Systems, Representation, and Arithmetic Hardware

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Deadline: two weeks from the date of issue — late submissions lose 10% per day unless a documented excuse is provided in advance (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.

2.1 Conceptual

Q1. Three schemes can encode negative numbers in n bits: sign-magnitude, one's complement, and two's complement. Explain, in terms of (i) how negation is performed and (ii) how addition behaves on a single adder, why two's complement is the scheme a hardware designer chooses. Your answer must name the specific defect of each of the two losing schemes (the property that rules each out for addition hardware).

Q2. In two's complement, a carry out of the most-significant bit is *not* the same thing as overflow. Explain the correct test for overflow, and give a concrete 8-bit example where a carry-out occurs but the result is *correct* (no overflow), and one where no carry-out occurs yet the result is *wrong* (overflow).

2.2 Numerical

Q3. (a) Convert 89_{10} to binary by repeated division by 2, showing the remainders. (b) Convert the binary result directly to hex by grouping bits, and check by converting your hex value back to decimal. (c) Represent -36 in 8-bit two's complement, showing the flip-and-add-1 steps.

Q4. Using the fixed-point **Q8.8** format (8 integer bits, 8 fraction bits), (a) write 6.25 and 1.5 in binary and add them bitwise to compute the sum; (b) write -2 in 8-bit two's complement (integer part only) and verify that $6.25 + (-2) = 4.25$ works by ordinary binary addition with the carry-out discarded.

2.3 Design

Q5. Using a 4-bit ripple-carry adder, trace the addition $+5 + (-2)$ in 4-bit two's complement, showing the carry chain bit-by-bit exactly as the lecture traced $7 + (-3)$. State the 4-bit result, the carry-out, and whether the result is correct in 4-bit two's complement (and why).

Q6. Design the subtraction path of the ALU: using the single-adder approach from the lecture, show the hardware sketch (one adder, one MUX, one forced carry-in) that computes $A - B$ as

$A + \bar{B} + 1$. For each of the two ALU operations **ADD** and **SUB**, state what the MUX selects and what the carry-in is. Explain why no separate subtractor is needed.